

INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

INGENIERÍA DE SISTEMAS

INGENIERÍA DE SOFTWARE II Y BIG DATA

**SESIÓN 4: DISEÑO DE ARQUITECTURA ANALITICA, MODELADO DE
REQUERIMIENTOS Y MÉTRICAS DE ESFUERZO**

**SANTIAGO HENAO JIMENEZ
KEVIN SMITH MONTENEGRO SANCHEZ
ALISSON NICOL PARRA LESMES
EDWARD ANDRES RUIZ AREVALO**

NOMBRE DE PROFE CHEVERE

21/08/2026

OCTAVO SEMESTRE

1. PATRONES DE ARQUITECTURA PARA PIPELINES DE DATOS

1.1. Arquitectura Limpia (Clean Architecture) y Modular

En el diseño de pipeline orientado a la detección de fraudes con tarjetas de crédito, se aplica el principio de separación de responsabilidades. Bajo este enfoque, los procesos de extracción, transformación/modelado y carga se organizan en módulos o clases completamente independientes entre sí.

¿Por qué se deben mantener los componentes aislados en este contexto?

- **Desacoplamiento estructural:** El dataset de fraude proviene de un archivo como `creditcard.csv` pero en el futuro podría migrar desde un almacenamiento local hacia un bucket de AWS S3 o una base de datos en la nube. Al aislar el proceso de extracción, cualquier cambio en la fuente de datos solo afecta al componente de ingesta, sin tocar las reglas de negocio ni el proceso de los datos.
- **Mantenibilidad y pruebas unitarias:** Dado que las transacciones legítimas superan ampliamente a las fraudulentas, el módulo de transformación requiere lógica especializada. Tenerlo aislado permite ejecutar pruebas unitarias con conjuntos de datos simulados en memoria RAM, sin necesidad de leer repetidamente el archivo completo de Kaggle.
- **Reusabilidad de código:** Las transformaciones matemáticas y de limpieza aplicadas a los componentes principales pueden estructurarse en métodos independientes, lo que permite reutilizarlos tanto en el pipeline de entrenamiento como en el de inferencia en tiempo real.

1.2. Flujo de datos unidireccional

FASE DEL FLUJO	COMPONENTE TÉCNICO	ESTADO DEL DATO
Ingesta	<code>extractor.py</code>	Almacenamiento secundario: El dataset crudo (<code>creditcard.csv</code>) reside de forma pasiva y persistente en el disco duro. Contiene transacciones con características anonimizadas mediante PCA, las columnas "Time", "Amount" y la variable objetivo "Class".
Procesamiento	<code>transformer.py</code>	Memoria principal: Los datos se cargan en un DataFrame de Pandas. En esta fase, el transformador normaliza las columnas de transacciones, gestiona el desbalance numérico y aplica el algoritmo analítico para identificar registros sospechosos.
Consumo	<code>loader.py</code>	Interfaz de salida: El DataFrame con las alertas de fraude y las métricas calculadas se consolida y exporta al entregable final.

1.3. Diseño de la interfaz del software

El pipeline está diseñado para ejecutarse de forma autónoma en un servidor de producción. Los parámetros y rutas se cargan automáticamente desde `config.py`, y la ejecución avanza de manera secuencial registrando cada evento y el porcentaje de fraude detectado en un archivo de registros.

2. INGENIERÍA DE REQUERIMIENTOS Y CASOS DE USO EN BIG DATA

2.1. Requerimientos Funcionales de Datos

ID	REQUERIMIENTO
RF1	El sistema debe detectar y eliminar las filas duplicadas del dataset
RF2	El sistema debe validar la ausencia de valores nulos en las 31 columnas antes de continuar el pipeline
RF3	El sistema debe escalar/normalizar las variables Amount y Time (rango 0–25,691.16 y 0–172,792 seg. respectivamente), ya que las variables V1–V28 ya están estandarizadas por PCA.
RF4	El sistema debe agrupar las transacciones por la variable Class y calcular estadísticos agregados (media, desviación estándar, conteo) de Amount por clase, para caracterizar el comportamiento fraude vs. no-fraude.
RF5	El sistema debe aplicar una técnica de balanceo de clases (undersampling, oversampling o SMOTE), dado el desbalance extremo (0.17% fraude).
RF6	El sistema debe particionar el dataset en entrenamiento y prueba de forma estratificada respecto a Class, para preservar la proporción de fraudes en ambos subconjuntos.
RF7	El sistema debe entrenar un modelo de clasificación binaria capaz de predecir Class (0 = legítima, 1 = fraude).
RF8	El sistema debe calcular métricas de evaluación distintas a accuracy (Precision, Recall, F1-score, AUC-PR), dado que el accuracy es engañoso con este desbalance.
RF9	El sistema debe permitir la ingesta de nuevos lotes de transacciones y clasificarlas con el modelo entrenado.
RF10	El sistema debe registrar (logging) toda transacción clasificada como fraude, incluyendo timestamp del procesamiento, para fines de auditoría.

2.2. Requerimientos No Funcionales (RNF) — Atributos de calidad

ID	CATEGORÍA	REQUERIMIENTO
RNF1	Rendimiento / Latencia	El pipeline completo (carga + limpieza + transformación) sobre las 284,807 filas debe ejecutarse en un tiempo máximo definido (ej. ≤ 5 minutos) en el entorno Colab/Codespaces.
RNF2	Consumo de memoria	El proceso no debe exceder el límite de RAM de la VM gratuita de Colab (~12-13 GB). El dataset ocupa ~67 MB en memoria como DataFrame de pandas, pero operaciones como balanceo (oversampling) pueden multiplicar ese tamaño, por lo que debe definirse un techo (ej. ≤ 2 GB de uso pico).
RNF3	Escalabilidad	El diseño debe permitir procesar volúmenes mayores (ej. si el dataset creciera a varios GB) mediante procesamiento por lotes (chunks) o migración a Spark/Dask sin rediseño total.

RNF4	Seguridad / Privacidad	Dado que V1–V28 son resultado de una transformación PCA (anonimización ya aplicada por el proveedor del dataset), el sistema debe garantizar que no se intente revertir esa transformación ni se exponga Amount/Time fuera del entorno controlado de análisis.
RNF5	Confiabilidad del modelo	El modelo debe alcanzar un Recall mínimo definido (ej. ≥ 0.85) sobre la clase minoritaria (fraude), ya que un falso negativo (fraude no detectado) tiene un costo mucho mayor que un falso positivo.
RNF6	Disponibilidad	En un escenario de producción (no en el prototipo de Colab), la clasificación de una transacción nueva debe responder en tiempo casi real (ej. < 1 segundo), aunque esto se documenta como requerimiento futuro, no del prototipo actual.
RNF7	Mantenibilidad / Reproducibilidad	El código del pipeline debe ser reproducible (versionado, semillas fijas en el muestreo/balanceo) para que los resultados sean auditables y comparables entre ejecuciones.
RNF8	Trazabilidad/Auditoría	El sistema debe registrar qué técnica de balanceo de clases y qué versión del dataset se usó en cada ejecución del pipeline, para poder reproducir o auditar exactamente cómo se entrenó el modelo que tomó una decisión de fraude.
RNF9	Portabilidad	el pipeline debe ejecutarse sin modificaciones sustanciales tanto en Google Colab como en GitHub Codespaces (mismas versiones de librerías clave como pandas, scikit-learn, imbalanced-learn), evitando dependencias específicas de un solo entorno.
RNF10	Usabilidad de resultados	Las métricas y visualizaciones generadas (matriz de confusión, curva Precision-Recall) deben presentarse de forma clara y estandarizada, de modo que un analista no técnico pueda interpretar si el modelo es apto para producción sin necesidad de leer código.

2.3. Casos de uso

- **CU1 – Preparar dataset para modelado:**
Actor: Científico de Datos. El sistema recibe el CSV crudo, elimina duplicados, valida nulos, escala variables y entrega un dataset limpio listo para split.
- **CU2 – Balancear y entrenar modelo de detección de fraude:**
Actor: Científico de Datos. El sistema toma el dataset limpio, aplica balanceo de clases, entrena el modelo y genera métricas (Precisión/Recall/F1/AUC-PR).
- **CU3 – Clasificar nueva transacción:**
Actor: Sistema de monitoreo transaccional (automatizado). Recibe una transacción nueva con sus 28 variables PCA + Amount + Time, la pasa por el modelo entrenado y devuelve una etiqueta (fraude / no fraude) con marca de tiempo para auditoría.

3. MÉTRICAS DE ESFUERZO, QOS Y QOE EN EL SOFTWARE

3.1. Métricas de Complejidad y Esfuerzo (Conceptos de Halstead y Shooman adaptados)

Las métricas de esfuerzo permiten estimar qué tan complejo resulta desarrollar, mantener y ampliar el software. En este proyecto, el sistema está construido para analizar transacciones financieras utilizando Pandas, por lo que el esfuerzo depende de la cantidad de operaciones aplicadas sobre los datos.

Aplicación de conceptos de Halstead

Halstead propone medir el software mediante operadores y operandos.

Para este caso:

Operadores (acciones realizadas sobre los datos):

- `read_csv()`
- `groupby()`
- `filter()`
- `corr()`
- `mean()`
- comparaciones (`==`, `>`, `<`)
- selección por columnas

Operandos (datos manipulados):

- Columnas (Time, Amount, Class, V1–V28)
- Valores numéricos
- DataFrames

Ejemplo simplificado del procesamiento:

```
df = pd.read_csv("creditcard.csv")
fraudes = df[df["Class"] == 1]
promedio = fraudes["Amount"].mean()
```

Aquí aumenta el esfuerzo porque el programa:

1. Carga datos.
2. Filtra registros.
3. Calcula métricas.

Mientras más transformaciones agreguen (detección automática, gráficos, Machine Learning), mayor será el volumen y complejidad.

Longitud y Volumen del Programa

Para este sistema:

- **Longitud estimada:** media–alta.
- **Volumen:** elevado debido al tamaño del dataset (284 mil registros).
- **Nivel de abstracción:** alto, porque Pandas simplifica operaciones complejas con pocas líneas.

Ejemplo:

- **Sin Pandas:**
for fila in datos:
 if fila["Class"] == 1:
 ...
- **Con Pandas:**
df[df["Class"] == 1]

Esto reduce esfuerzo de desarrollo y mantenimiento.

Adaptación del concepto de Shooman

Shooman relaciona complejidad con mantenimiento.

En este proyecto:

- Arquitectura modular → menor dificultad de mantenimiento.
- Separación por módulos → mayor escalabilidad.
- Inclusión futura de Machine Learning → crecimiento controlado.

Conclusión: el uso de Pandas disminuye la complejidad estructural y mejora mantenibilidad.

3.2. Calidad de Servicio (QoS)

La QoS (Quality of Service) mide el rendimiento técnico del software.

Para este proyecto se consideran:

a) Tiempo de procesamiento

Probando operaciones básicas sobre el archivo:

- Tiempo aproximado de análisis inicial: 0,07 segundos
(lectura de datos, filtrado y cálculo de indicadores simples).

Esto indica que el sistema responde rápidamente para análisis exploratorios.

b) Uso de memoria

El dataset ocupa aproximadamente:

- 67,36 MB de memoria

Por tanto:

TAMAÑO DEL ARCHIVO	RECURSO RECOMENDADO
Hasta 100 MB	8 GB RAM
100–500 MB	16 GB RAM
Más de 500 MB	procesamiento optimizado

c) Escalabilidad

La arquitectura modular permite:

- agregar nuevas métricas;
- incorporar visualización;
- integrar modelos predictivos;

- dividir procesamiento por etapas.

Indicadores QoS propuestos

INDICADOR	RESULTADO ESPERADO
Tiempo de respuesta	< 1 segundo
Uso de memoria	≤ 70 MB
Disponibilidad	Alta
Escalabilidad	Modular

3.3. Calidad de la Experiencia (QoE)

La QoE (Quality of Experience) evalúa cómo perciben el sistema quienes usan los resultados.

En este caso los usuarios finales serían:

- analistas financieros;
- supervisores;
- tomadores de decisiones.

Factores evaluados

Fidelidad de los datos

Los indicadores deben representar correctamente los fraudes detectados.

Resultado observado:

- Fraudes reales encontrados: 492 transacciones
- Representan: 0,17 % del total

Esto evidencia un escenario real donde el fraude es poco frecuente.

Claridad

Los resultados deben mostrarse mediante:

- tablas;
- gráficos;
- indicadores simples;
- alertas.

Ejemplo:

Total transacciones: 284.807

Fraudes detectados: 492

Porcentaje fraude: 0,17 %

Velocidad: El usuario espera resultados casi inmediatos.

Objetivo: generación de indicadores < 2 segundos.

Indicadores QoE propuestos

INDICADOR	OBJETIVO
Comprensión	Alta
Precisión percibida	Alta
Rapidez percibida	Alta
Satisfacción	$\geq 90 \%$

4. LABORATORIO AUTÓNOMO: MAQUETACIÓN DEL PROYECTO

4.1. Actividad Práctica Grupal:

Análisis de Fraudes con Tarjetas de Crédito

Definición de Roles y Responsabilidades

NOMBRE	ROL	RESPONSABILIDAD
Santiago Henao	Arquitecto de Software	Diseñar la estructura y arquitectura del sistema.
Kevin Montenegro	QA / Documentación	Validar la calidad del sistema y documentar el proyecto.
Nicol Parra	Especialista en Visualización	Crear gráficos y reportes para el análisis de fraudes.
Edward Ruiz	Ingeniero ETL/Pandas	Preparar y transformar los datos para el análisis.

Presentación de Roles

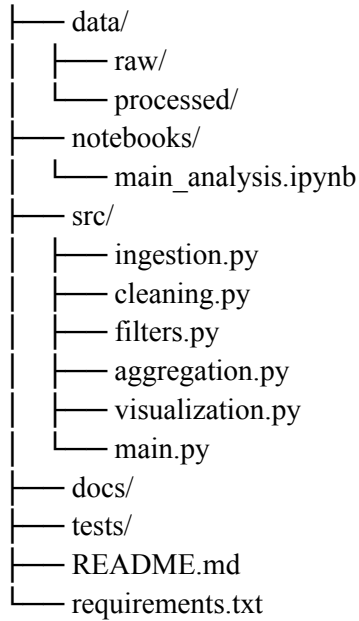
- Yo soy Santiago Henao (Arquitecto de Software) y mi responsabilidad es Diseñar la estructura y arquitectura del sistema.
- Yo soy Kevin Montenegro (QA / Documentación) y mi responsabilidad es Validar la calidad del sistema y documentar el proyecto.
- Yo soy Nicol Parra (Especialista en Visualización) y mi responsabilidad es Crear gráficos y reportes para el análisis de fraudes.
- Yo soy Edward Ruiz (Ingeniero ETL/Pandas) y mi responsabilidad es Preparar y transformar los datos para el análisis.

4.2. Diseño del Diagrama de Arquitectura



4.3. Estructura del Repositorio GitHub

fraud-detection-analysis/



Comentarios Iniciales para los Archivos

ingestion.py

```
"""
Módulo encargado de la carga del dataset de fraudes.
Funciones:
- Leer archivos CSV.
- Validar existencia de archivos.
- Retornar DataFrame principal.
"""
```

cleaning.py

```
"""
Módulo de limpieza y transformación de datos.
Funciones:
- Eliminar duplicados.
- Gestionar valores nulos.
- Convertir tipos de datos.
"""
```

filters.py

```
"""
Módulo de filtrado.
Funciones:
- Filtrar transacciones fraudulentas.
- Filtrar por montos.
- Filtrar por fechas.
"""
```

aggregation.py

"""

Módulo de agregación y análisis.

Funciones:

- Conteo de fraudes.
- Promedio de montos.
- Estadísticas descriptivas.

"""

visualization.py

"""

Módulo de visualización.

Funciones:

- Histogramas.
- Gráficos de barras.
- Gráficos de distribución.

"""

main.py

"""

Archivo principal del proyecto.

Orquesta el flujo:

1. Ingesta.
2. Limpieza.
3. Filtros.
4. Agregación.
5. Visualización.

"""

4.4. Descripción para la Tutoría del Docente

El objetivo es desarrollar una aplicación de análisis de datos con Pandas para identificar patrones de fraude en transacciones con tarjetas de crédito. La arquitectura propuesta es modular, escalable y facilita el mantenimiento y futuras ampliaciones, incluyendo modelos de Machine Learning.